

Analisis tecnico del ataque compuesto sobre wallet local

Documento interno de auditoria tecnica

26 de mayo de 2026

Índice

1. Resumen ejecutivo	3
2. Objetivo del documento	3
3. Estado actual de PoC	3
3.1. Estado de integracion en Zallet	4
4. Fuentes usadas dentro de esta branch	6
5. Hipotesis principal	7
5.1. Capacidades del atacante	7
6. Mapa del ataque compuesto	7
6.1. Vista de alto nivel	7
7. Fase 1: admision de estado no validado	7
7.1. Bug 1A: decrypt_and_store_transaction no es un validador de consenso . . .	7
7.2. Bug 1B: compact scanning puede promover outputs sin validacion local completa	7
7.3. Bug 1C: confusion de contexto ZIP-212	7
7.4. Por que esta fase es importante	8
8. Fase 2: persistencia sin rebinding suficiente	8
8.1. Bug 2A: nota persistida sin recheck de commitment	8
8.2. Bug 2B: nullifier y tree position caller-supplied	8
8.3. Bug 2C: memo y output index no reatados al mismo output cifrado	8
8.4. Bug 2D: upserts sticky	8
8.5. Por que esta fase es importante	8
9. Fase 3: corrupcion de intent en outputs enviados	9
9.1. Bug 3A: PCZT output intent fields son wallet-bound	9
9.2. Bug 3B: full UA mutable despues de elegir receiver concreto	9
9.3. Bug 3C: sender recovery solo protege memo, no recipient/value/account	9
9.4. Consecuencia global de la fase 3	9
10.Fase 4: amplificacion en spendability local	9
10.1. Como las fases anteriores contaminan gasto local	9
10.2. Tipo de fallo esperado	9
11.Fase 5: amplificacion por reorg	9
11.1. Bug 5A: deep rewind no limpia todo al target pedido	9
11.2. Por que esto agrava la cadena	9

12.Fase 6: DoS como cierre de la cadena	10
12.1. Bug 6A: valores fuera de rango paniquean	10
12.2. Bug 6B: compact metadata malformada paniquea scanning	10
12.3. Valor ofensivo de esta fase	10
13.Ataques compuestos mas prometedores	10
13.1. Cadena A: input no validado + persistencia + spendability	10
13.2. Cadena B: PCZT mutable + historial falso + persistencia	10
13.3. Cadena C: estado contaminado + deep rewind	10
14.Impacto compuesto final	11
15.Que partes queremos PoCear primero	11
15.1. Estado actual del target SQLite	11
15.2. Targets obligatorios	11
15.3. Razon de esos targets	11
16.Plan de PoC recomendado	11
16.1. PoC 1: version “minima fuerte”	11
16.2. PoC 2: version “PCZT + history corruption”	12
16.3. PoC 3: version “reorg amplification”	12
16.4. PoC 4: version “panic as amplifier”	12
17.Que no debemos exagerar en la PoC	12
18.Resumen final	12

1. Resumen ejecutivo

Este documento reemplaza el informe anterior centrado en un solo bug de PCZT. La tesis actual es mas ambiciosa:

No estamos frente a un unico bug aislado, sino frente a una cadena de bugs wallet-side que se pueden combinar para producir un impacto local mayor: historial falso, recipient intent falso, estado spendable corrupto, persistencia de metadata no revalidada, fallos tardios al gastar y denegacion de servicio durante recovery / scan / rescan.

La conclusion central es:

Lo que no afirmamos. No confirmamos robo directo de fondos on-chain ni bypass de consenso.

Lo que si afirmamos. Si confirmamos una cadena plausible y fuerte de **corrupcion de estado de wallet local**.

El ataque compuesto mas fuerte que hoy podemos sostener tecnicamente es:

primero, el wallet acepta o procesa datos shielded no plenamente validados; despues, persiste campos de nota y metadata auxiliar sin rebindearlos suficientemente al commitment y al output real; mas adelante, en flujos PCZT, sigue confiando en metadata mutable para recipient, cuenta, valor y direccion visible; luego, tras un deep rewind, puede preservar parte de ese estado criptografico inconsistente; y finalmente varias ramas permiten panic antes de que una correccion por rescan o reprocesamiento limpie el estado.

Eso lleva a un impacto agregado tipo:

sent history falso, received history semantica o estructuralmente inconsistente, memo u output index mal asociados, spendability local incorrecta, witness o nullifier state persistiendo mas de lo debido, fallos tardios al construir spends y DoS operacional.

2. Objetivo del documento

Este documento persigue cuatro metas. Primero, describir el **ataque compuesto** que queremos demostrar con PoC. Segundo, explicar **muy detalladamente** los bugs chicos que lo habilitan. Tercero, dejar claro cuales partes son **hallazgos confirmados** y cuales son **composicion / amplificacion**. Cuarto, fijar el **plan de implementacion de PoC** en la misma forma usada en la branch anterior: como tests en `zcash_client_backend`, `zcash_client_memory` y, para las rutas wallet-side relevantes, tambien en `zcash_client_sqlite`.

3. Estado actual de PoC

La cadena compuesta ya no es solo un diseno teorico. En esta branch quedaron PoC ejecutables para dos rutas ya demostradas. La primera es la **corrupcion de intent en PCZT**: hay tests backend para Sapling y Orchard, tests sobre `zcash_client_memory` y ahora tambien tests sobre `zcash_client_sqlite` que muestran que la sent history puede persistir un recipient falso o reutilizar una reclasificacion falsa de output externo como estado wallet-interno. La segunda es el **mismatch de contexto ZIP-212**: hay un test backend que demuestra que la misma transaccion puede ser tratada como wallet-owned o no segun el contexto de cadena que se pase a `decrypt_transaction`; y esa misma superficie wallet-visible ya quedo revalidada tambien en `zcash_client_sqlite` para la ruta de sent history.

En otras palabras, hoy ya tenemos demostradas por tests la ruta “PCZT mutable → historial falso”, la ruta “PCZT mutable → reclasificación de output externo como `InternalAccount`”, la ruta “reclasificación falsa → resumen historico reutiliza esa semantica interna”, la ruta “contexto legacy → aceptación wallet-side bajo semantica equivocada” y la ruta “contexto legacy → sent history visible” tanto para el wallet in-memory como para `zcash_client_sqlite` cuando la transaccion se interpreta como *outgoing* bajo el contexto equivocado.

Ademas, esta branch ya contiene una **PoC compuesta sobre el mismo wallet** que une ambas familias en una sola historia operativa. Primero, el wallet acepta una transaccion Sapling legacy por falta de contexto de cadena y la expone por `get_sent_outputs()`. Despues, en ese mismo wallet, una segunda transaccion creada por el flujo proposal → PCZT → prove → sign → extract queda reclasificada como `InternalAccount` aunque el output comprometido siga siendo externamente recoverable. Finalmente, el mismo wallet queda en un estado donde la tx legacy sigue mostrando recipient externo por `get_sent_outputs()`, mientras que la tx PCZT ya no muestra ningun recipient externo por esa misma API.

El efecto compuesto mas fuerte que hoy ya esta demostrado por test es que, cuando ambas corrupciones coexisten en el mismo wallet, `get_tx_history()` deja de producir una vista global consistente y falla identificando la transaccion legacy admitida por el path sin contexto.

Esa ya no es la ultima etapa demostrada. Hoy la branch tambien contiene una **variante ejecutable de escalada a High**: se toma ese mismo wallet ya corrompido por las dos familias previas, se lo deja en un estado todavia suficientemente vivo como para seguir escaneando y luego un unico bloque compacto malformado en la siguiente altura hace panicar el `scan_cached_blocks()` normal.

En la variante actual, el campo malformado elegido es `prev_hash` con longitud invalida, de modo que el panic aparece durante la verificacion de continuidad de bloques en el scanner y no por un hook artificial del test.

La parte que **todavia no** esta ejecutable en `zcash_client_memory` es la amplificacion por reorg profundo, porque ese target todavia no implementa `rewind_to_chain_state`.

3.1. Estado de integracion en Zallet

La PoC ya no vive solamente dentro de `librustzcash`. Tambien quedo integrada en el repo de `Zallet` en dos niveles.

Variante white-box. La primera variante es **white-box fuerte** sobre el engine de sync de `Zallet`. Ejerce `initialize` y el loop real de `steady_state` sobre un `MockChain` y demuestra panic cuando el siguiente bloque malformado entra por la ruta normal de sync.

Variante black-box. La segunda es una **black-box de producto** que ejecuta `zallet start` contra un validator RPC stub minimo y demuestra que el proceso termina de manera fallida por input malicioso proveniente del origen normal de datos de cadena.

Hay una diferencia tecnica importante entre ambas. La variante white-box sobre el loop de sync conserva la idea del `CompactBlock` malformado. La variante black-box por subprocess no usa el mismo `prev_hash` truncado, porque en la ruta `FetchService -> BlockCache -> CompactBlock` Zaino reconstruye ese campo desde el bloque crudo. Por eso, en `zallet start`, la superficie efectiva elegida fue `z_getsubtreesbyindex`: `Zallet` hace `unwrap()` sobre roots devueltos por el validator durante `initialize`, y un root malformado produce el crash de producto.

Este punto es importante para disclosure: la composicion exacta del bug puede variar entre capas, pero el mensaje operativo sigue siendo el mismo: **un wallet local real puede caerse por input malicioso proveniente de su fuente normal de sync.**

La integracion en `Zallet` ya subio un paso mas: no solo se confirmo un crash de producto aislado, sino tambien **persistencia del fallo tras restart**. Hoy ya hay una PoC donde:

1. se inicializa una wallet real;
2. se la hace arrancar contra el mismo validator RPC stub malicioso;
3. `zallet start` falla;
4. sin reparar nada en disco ni cambiar de backend, se vuelve a correr `zallet start`;
5. y la wallet vuelve a fallar por la misma condicion.

Eso no prueba robo ni consenso, pero si refuerza mucho el framing de *freezing* operacional: el usuario no enfrenta un crash accidental unico, sino un estado donde la wallet no puede progresar autonomamente mientras persista la misma fuente de sync maliciosa.

Ademas, la integracion de Zallet ya mejoro en dos direcciones adicionales que suben la calidad probatoria del impacto. Por un lado, ya existe una variante white-box donde el wallet tiene balance transparente visible por `WalletSummary` antes de entrar en el crash del loop de `steady_state`. Por otro lado, ya existe una variante separada donde la tarea real de `recover_history` paniquea al procesar bloques historicos malformados, en vez de depender del arranque o del steady-state loop.

Eso fortalece dos partes distintas del argumento:

1. que el crash puede afectar una wallet que ya tiene estado economico visible y no solo una wallet “vacía”;
2. que el mismo defecto no solo rompe el sync normal, sino tambien una ruta de recuperacion que el usuario razonablemente intentaria usar para volver a un estado sano.

Es importante describir con precision el alcance actual de estas dos mejoras:

- el caso de **balance visible antes del crash** ya esta probado en una variante white-box del loop real de `steady_state`, y ademas ya existe una variante black-box donde `zallet start` falla contra el validator malicioso despues de que el `wallet.db` fue presembrado con un UTXO wallet-owned reconocido;
- el caso de **crash en recovery historico** ya esta probado en una variante white-box de la tarea real de `recover_history`, y ademas ya existe una variante black-box por subprocess donde `zallet start` termina prematuramente al entrar por una historia historica real servida por un validator malicioso;
- la variante black-box por subprocess de `zallet start` hoy sigue demostrando la caida de producto y la persistencia tras restart; de las dos propiedades adicionales, ahora ya incorpora tanto la de estado wallet-owned previo al crash como la de historia historica real. Lo que cambia entre variantes es **como** se manifiesta el fallo.

En otras palabras, la evidencia actual ya demuestra las tres cosas fuertes por separado:

1. crash o terminacion prematura de producto;
2. persistencia del fallo tras restart;
3. impacto sobre wallet con estado economico visible;
4. impacto sobre recovery historico.

Lo que todavia queda como mejora opcional, y no como condicion para sostener el reporte, seria reunir todas esas propiedades dentro de una unica PoC black-box de producto con una semantica de fallo uniforme. Hoy la variante historica por subprocess ya existe, pero se observa como terminacion prematura de `zallet start` porque el supervisor actual descarta el `Result` de las tareas de sync y apaga el proceso en cuanto alguna de ellas termina.

Ese paso de unificacion tambien quedo alcanzado. Hoy ya existe una variante black-box de producto donde, dentro del mismo escenario general:

1. la wallet ya tiene estado `wallet-owned` visible en disco;
2. el trabajo pendiente real cae en historia historica;
3. el validator malicioso provoca la terminacion de `zallet start`;
4. un segundo `zallet start` vuelve a terminar de manera prematura bajo la misma fuente maliciosa;
5. e incluso una reparacion local agresiva del estado wallet no evita la terminacion mientras persista el mismo backend.

Con eso, las propiedades que antes estaban demostradas por piezas quedaron ya demostradas tambien en una PoC black-box unificada:

- estado `wallet-owned` visible;
- historia historica real;
- terminacion de producto;
- persistencia tras restart;
- freezing practico frente a reinicios y frente a una reparacion local del estado.

La unica reserva que sigue siendo importante es semantica: **la variante historica por subprocess se manifiesta como terminacion prematura del proceso, no necesariamente como exit code no-cero**, porque el supervisor actual trata la salida de una tarea ongoing como condicion de apagado aunque el error no se propague como codigo de salida.

Pero desde el punto de vista del impacto operacional sobre el usuario, esa reserva ya no cambia demasiado la lectura: la wallet queda en una condicion de *freezing* practico mientras siga dependiendo de la misma fuente de sync maliciosa.

4. Fuentes usadas dentro de esta branch

Los dos documentos auxiliares de esta branch que guiaban la PoC anterior son los siguientes. Uno es `LIBRUSTZCASH_AUDIT_FINDINGS.md`. El otro es `LIBRUSTZCASH_PCZT_AUDIT.md`.

De ellos extraemos dos decisiones metodologicas. La primera es que la PoC debe vivir como **tests ejecutables** dentro del repo. La segunda es que el lugar correcto para empezar es `zcash_client_backend` y `zcash_client_memory`. Eso ya dio resultado, y desde ahi la misma familia de PoC relevantes pudo bajarse tambien a `zcash_client_sqlite` para confirmar que la corrupcion no era un artefacto del backend mock ni del store in-memory.

5. Hipotesis principal

La hipotesis de trabajo para la PoC compuesta es:

Un atacante con capacidad de introducir estado no plenamente validado o metadata mutable en un wallet local puede conseguir que el wallet almacene y reutilice una vista incorrecta del estado shielded, y que esa corrupcion se propague a historial, recipient intent, spendability local y recovery / rescan, con posibilidad adicional de DoS.

5.1. Capacidades del atacante

No hace falta que el atacante tenga *todas* estas capacidades juntas. El documento agrupa varias rutas. Las capacidades relevantes son un servidor remoto malicioso o corrupto que alimenta compact blocks, input local de recovery o import que llama APIs wallet-side sobre transacciones no plenamente validadas, un coordinador o tooling step malicioso dentro de un flujo PCZT, o un momento de deep rewind o rescan sobre estado ya contaminado.

6. Mapa del ataque compuesto

6.1. Vista de alto nivel

La cadena completa puede verse asi:

admission → persistence → intent corruption
→ spendability corruption → reorg amplification → panic/doS

Cada fase puede ocurrir o no. No hace falta completar toda la cadena para tener impacto relevante, pero el peor caso local aparece cuando varias fases se encadenan.

7. Fase 1: admision de estado no validado

7.1. Bug 1A: decrypt_and_store_transaction no es un validador de consenso

Hallazgo confirmado. decrypt_and_store_transaction puede decriptar y persistir salidas wallet-relevant sin primero probar que la transaccion sea consenso-valida.

Consecuencia. Entra al wallet un estado que semantica y estructuralmente se trata como si ya hubiese pasado una frontera de validacion mas fuerte de la que realmente paso.

7.2. Bug 1B: compact scanning puede promover outputs sin validacion local completa

Hallazgo confirmado. El path de compact scanning puede elevar outputs shielded a un estado local que parece wallet-owned o spendable-looking, sin verificacion local de proofs ni signatures.

Consecuencia. Esto no implica aceptacion de consenso, pero si implica que el wallet puede formar opiniones locales fuertes sobre ownership y spendability antes de una validacion completa.

7.3. Bug 1C: confusion de contexto ZIP-212

Hallazgo confirmado. Si falta contexto de altura, decrypt_transaction puede terminar aplicando el modo de enforcement pre-ZIP-212 (Off) a transacciones no minadas actuales.

Consecuencia. El wallet puede aceptar plaintext semantics pre-ZIP-212 en un contexto donde consenso actual no las aceptaria.

7.4. Por que esta fase es importante

La Fase 1 define la frontera de ingreso:

el wallet empieza a trabajar con datos shielded que son decryptable o parseable, pero todavia no estan suficientemente amarrados a la semantica de consenso que el resto del sistema informalmente asume.

8. Fase 2: persistencia sin rebinding suficiente

8.1. Bug 2A: nota persistida sin recheck de commitment

Hallazgo confirmado. Una vez que una `Received*Output` cruza a storage, los componentes de nota se persisten sin recomputar y comparar contra el `cmu/cmx` publicado.

Esto rompe la idea deseable:

$$\text{stored note state} \iff \text{published commitment}$$

porque la equivalencia deja de revalidarse en la frontera de persistencia.

8.2. Bug 2B: nullifier y tree position caller-supplied

Hallazgo confirmado. Nullifier y commitment-tree-position pueden aceptarse y persistirse como metadata provista por el caller, en vez de rederivarse o cross-checkearse de forma canonica.

Consecuencia. El wallet puede construir una vision local de spentness o witness-availability basada en metadata no suficientemente reatada al note real.

8.3. Bug 2C: memo y output index no reatados al mismo output cifrado

Hallazgo confirmado. Memo y output/action index pueden persistirse como piezas independientes sin reafirmar que siguen apuntando al mismo output ciphertext-bearing.

8.4. Bug 2D: upserts sticky

Hallazgo confirmado. Ciertas asociaciones wallet-side en SQLite pueden quedar “pegadas” bajo semantica de upsert. En particular, la confirmacion reciente sobre `zcash_client_sqlite` mostro una causa concreta de divergencia: `sent_notes` retenia `to_address` y `to_account_id` previos aunque una actualizacion posterior reclasificara el output.

Ese detalle importaba directamente para las PoC de reclasificacion falsa de output externo como `InternalAccount`, porque hacia que SQLite siguiera mostrando recipient externo en `sent history` y `tx history` incluso cuando el estado ya habia sido reescrito como wallet-interno. Corregido ese punto, las PoC relevantes de reclasificacion y reutilizacion de historial ya pasan sobre `zcash_client_sqlite`.

8.5. Por que esta fase es importante

La Fase 2 transforma un problema de admision transitoria en:

corrupcion persistente de estado wallet

Una vez persistido el estado, el sistema puede reconstruir objetos internamente plausibles aunque el anchor final de verdad on-chain ya no sea el que manda.

9. Fase 3: corrupcion de intent en outputs enviados

9.1. Bug 3A: PCZT output intent fields son wallet-bound

Hallazgo confirmado. `user_address` y la metadata propietaria de output no estan protegidos por consenso, sighash ni binding signatures.

pero el wallet los sigue tratando como si fuesen verdad fuerte luego de extraer la transaccion.

9.2. Bug 3B: full UA mutable despues de elegir receiver concreto

Hallazgo confirmado. Una vez que proposal/build eligio un receiver concreto de una UA, el resto de la UA ya no esta tx-bound; aun asi, la capa de extraction/storage puede guardar y mostrar la UA completa desde metadata mutable.

9.3. Bug 3C: sender recovery solo protege memo, no recipient/value/account

Hallazgo confirmado. En ciertos paths de sent-output reconstruction desde PCZT, sender recovery se usa para memo, mientras recipient, value y account salen primero de metadata mutable.

9.4. Consecuencia global de la fase 3

Esta fase no cambia la tx on-chain, pero si cambia a quien cree el wallet que le pagaste, que full UA cree que se uso, que cuenta interna cree que estuvo involucrada y que valor cree que esta asociado a cierto output sent-history.

10. Fase 4: amplificacion en spendability local

10.1. Como las fases anteriores contaminan gasto local

Si el wallet ya persistio note internals no revalidados, nullifier metadata caller-supplied, tree positions no rebindeadas y recipient o account semantics falsas, entonces las capas posteriores pueden reportar spendable balance equivocado, intentar generar witness sobre posiciones inconsistentes, considerar notas como spendables hasta que falle una verificacion tardia y generar propuestas o builds que solo fallan mas adelante.

10.2. Tipo de fallo esperado

No esperamos aqui un gasto on-chain valido con nullifier mal ligado. Lo que esperamos es **late failure, user-visible inconsistency y wallet-state corruption**.

11. Fase 5: amplificacion por reorg

11.1. Bug 5A: deep rewind no limpia todo al target pedido

Hallazgo confirmado. En deep rewind, el wallet puede preservar witness state, mined tx state y nullifier-derived spentness por encima del target solicitado, hasta el pruning floor.

11.2. Por que esto agrava la cadena

Si el estado ya venia contaminado por fases previas, el deep rewind puede hacer que esa contaminacion dure mas tiempo, mezclar estado “viejo pero preservado” con scan queue “nueva pero aun no reparada” y reforzar decisiones locales de spendability sobre una base criptografica inconsistente.

12. Fase 6: DoS como cierre de la cadena

12.1. Bug 6A: valores fuera de rango paniquean

Hallazgo confirmado. Ciertos note values out-of-range pueden paniquear al convertir a Zatoshis.

12.2. Bug 6B: compact metadata malformada paniquea scanning

Hallazgo confirmado. Txid, nullifiers y otros campos compact protobuf malformados pueden disparar panic en scanning.

12.3. Valor ofensivo de esta fase

Si queremos una PoC fuerte de impacto local, el DoS es valioso porque puede cortar el flujo de autoreparacion por rescane, dejar la base o el estado intermedio en un punto donde el usuario solo ve que “el wallet esta roto” y convertir corrupcion semantica en problema operacional visible.

13. Ataques compuestos mas prometedores

13.1. Cadena A: input no validado + persistencia + spendability

1. alimentar al wallet una tx decryptable pero no plenamente validada;
2. lograr que se persista;
3. observar que metadata de nota / nullifier / posicion no queda suficientemente reatada;
4. intentar un path que use ese estado como spendable;
5. demostrar fallo tardio o inconsistencia local visible.

13.2. Cadena B: PCZT mutable + historial falso + persistencia

1. crear un PCZT valido;
2. mutar metadata output-side;
3. extraer y almacenar;
4. probar que el wallet guarda recipient/value/account/history distinto del comprometido por la tx;
5. intentar que esa asociacion falsa sobreviva reprocesamiento local.

13.3. Cadena C: estado contaminado + deep rewind

1. introducir estado wallet-side inconsistente;
2. forzar deep rewind;
3. observar que parte del estado criptografico sobrevive arriba del target;
4. probar que spendability o history siguen reflejando ese estado mixto.

14. Impacto compuesto final

La forma mas honesta de describir el impacto es:

No tenemos un exploit de theft on-chain. Tenemos una **cadena de integridad y disponibilidad de wallet local** que puede producir:

historial falso, recipient intent falso, semantica de output errada, resúmenes históricos de transacción que heredan esa misma reclasificación falsa, balances o spendability local equivocadas, fallos tardíos al gastar y crash durante scanning, recovery o reprocesamiento.

Por eso la severidad correcta del compuesto es:

High para wallet local, **No Critical** y **No direct funds theft confirmed**.

15. Que partes queremos PoCear primero

15.1. Estado actual del target SQLite

`zcash_client_sqlite` ya no está bloqueado para las rutas principales que importan en esta historia. Hoy ya confirma:

- sent history falso por metadata mutable de PCZT;
- reclasificación falsa de output externo como `InternalAccount`;
- reutilización de esa reclasificación en superficies de tx history;
- y la ruta legacy-context → sent history visible.

15.2. Targets obligatorios

Las PoC deben hacerse del mismo modo que la investigación previa: como tests en backend (`zcash_client_backend`), como tests en memory (`zcash_client_memory`) y, cuando se trate de persistencia e historial wallet-side concreto, también sobre `zcash_client_sqlite`.

15.3. Razon de esos targets

`zcash_client_backend` nos deja demostrar el bug en la capa lógica o API. `zcash_client_memory` nos deja demostrar persistencia y comportamiento wallet-side con iteración rápida. Y `zcash_client_sqlite` ya nos deja confirmar que la misma corrupción alcanza también al store relacional real y a sus superficies SQL de sent history / tx history.

16. Plan de PoC recomendado

16.1. PoC 1: version “minima fuerte”

Objetivo. Demostrar que una tx o estado shielded no plenamente validado puede ser aceptado por APIs wallet-side bajo semántica de contexto equivocada y, desde ahí, servir como base para un flujo compuesto de corrupción de estado.

Forma. Hoy está implementada como test en `zcash_client_backend` y ya está enganchada con una PoC wallet-visible en `zcash_client_memory`.

16.2. PoC 2: version “PCZT + history corruption”

Objetivo. Demostrar recipient, value, account e history corruption a partir de metadata mutable PCZT en un flujo local end-to-end.

Esta PoC ya quedo implementada y validada en esta branch, y sigue siendo el lugar mas natural para profundizar. La infraestructura conceptual ya existe; backend y `zcash_client_memory` muestran la misma distincion entre recipient comprometido y recipient persistido; y el ataque compuesto puede apoyarse sobre esta misma base para sumar mas semantica errada, no solo recipient.

16.3. PoC 3: version “reorg amplification”

Objetivo. Demostrar que, una vez contaminado el estado local, un deep rewind no lo limpia totalmente y deja spendability o witness state en condicion inconsistente.

Estado actual. Esta linea esta documentada y confirmada por auditoria, pero **bloqueada en memory** por falta de implementacion de `rewind_to_chain_state`. Por lo tanto debe tratarse como siguiente etapa compuesta, no como prerequisite inmediato.

16.4. PoC 4: version “panic as amplifier”

Objetivo. Cerrar la historia mostrando que ciertas entradas malformadas pueden evitar que el wallet se autorepare mediante rescan o recovery.

17. Que no debemos exagerar en la PoC

Para que la PoC sea solida, no conviene afirmar robo directo de fondos, bypass de consenso ni doble interpretacion on-chain de una misma tx.

La PoC tiene que concentrarse en lo que si esta soportado por los hallazgos: corrupcion de estado wallet local, persistencia de metadata o note state mal reatada, spendability local equivocada, historial o intent falsos y DoS.

18. Resumen final

El viejo documento estaba bien para un solo bug PCZT. Ya no alcanza.

La imagen tecnica correcta hoy es la siguiente. `librustzcash` tiene varios bugs wallet-side medianos o bajos, y varios de ellos comparten la misma debilidad: confiar demasiado en datos no plenamente validados o no cryptographically rebound. Al combinarlos, el impacto real sube. El mejor target de demostracion es un **wallet local**, y la forma correcta de PoCearlo en esta branch sigue siendo con tests en backend y memory, aceptando que la parte de reorg hoy no puede cerrarse en memory sin implementar primero soporte faltante del target.

Ese es el ataque sobre el que conviene construir la siguiente tanda de PoC compuesta, arrancando por las rutas que ya estan demostradas y encadenandolas antes de intentar cubrir la parte de reorg.